LSE DEPARTMENT OF STATISTICS × SKY · ST498 CAPSTONE P13

MA-FDSS

An Empirical Analysis of Multi-Agent Systems for Football Data Analytics and Performance Prediction

MSc Data Science Researchers

Ethan Bensadoun · Lorenzo Ravera · Till Tornieporth

Supervisors

Dr Marcos Barreto (LSE) · Dr Huma Lodhi (Sky)

September 2026

Research Focus and Objectives

When is a multi-agent architecture the right choice for grounded football analytics?

01 Autonomy

Does a Data Analyst agent do better fully controlled by SQL templates, semi-controlled, or writing free SQL, in and out of template coverage?

02 Orchestration vs tools

Once tool access and schema context are held constant, does multi-agent orchestration add accuracy over a single tool-using LLM?

03 Memory

What does a persistent memory layer contribute to accuracy, latency and token cost, on isolated queries and across conversational turns?

04 Prediction

Can a calibrated match-outcome model, called as an agent tool, beat market-implied and naive baselines?

PART 1

Data & knowledge base

Six providers into one queryable layer, and the four rules that layer imposes on everything downstream.

PART 2

System & experiments

LangGraph orchestration, three degrees of analyst autonomy, two single-LLM baselines, a frozen golden set.

PART 3

Memory & prediction

Four memory stores, a 55-session ablation, and a calibrated match-outcome model.

Three parts. The architecture question runs through all of them.

Where we sit in the literature

Multi-agent LLM systems

Li et al. (2024) survey the field; Yang et al. (2024) locate the advantage in decomposition and specialisation; Ye et al. (2025) generate executable systems in one inference; Gulli (2025) systematises 21 agentic design patterns.

Our Manager / Analyst / Visualizer pipeline, bounded retry and rule-based Eval agent instantiate those patterns. The gap is evaluation: general-purpose QA benchmarks that separate neither orchestration nor its cost.

Agentic systems for football

SoccerAgent (Rao et al., 2025) contributes SoccerWiki, a ~10K multimodal QA benchmark, and a decomposing multi-agent system that beats MLLM baselines.

Its target is multimodal understanding; ours is structured analytical querying plus a calibrated predictive tool. It omits the equal-tools ablation, the autonomy sweep and the memory experiment.

Match-outcome prediction

Constantinou (2019) on dynamic ratings; Wong et al. (2024) stack classifiers, LightGBM and a CNN on the EPL against bookmaker odds; Bunker & Thabtah (2019) on chronological splits; Wheatcroft & Sienkiewicz (2021) on calibration-sensitive scoring.

Motivates our pre-kickoff features, out-of-sample testing, Brier and log loss, and isotonic calibration. Throughout, the predictor is evaluated in isolation, never exposed as a callable tool. Part 3 returns to it.

Memory and retrieval

Gulli (2025) separates short-term session context from a persistent, browsable layer queried by RAG. Huyen (2025) warns that semantic caching needs high-quality embeddings, or it retrieves the wrong cached answer.

Directly motivates our conversation / SQL-cache / corrections split, evaluating each store separately, and the safety analysis of similarity-keyed reuse in Part 3.

The evidence for multi-agent systems never isolates orchestration from tool access, nor prices it.

What we found

87% = 87%

MAS and a single LLM with the same tools tie on 30 questions (26/30). McNemar p = 1.0 — at 1.5× the tokens and 2× the latency.

70% → 87%

Single LLM, raw tables pasted into the prompt vs. tool access. Every question it loses is one whose table was too large to inline.

6% → 94%

Out-of-coverage accuracy, SQL templates vs. free SQL (Opus 4.8). Templates cannot answer what they were not written for.

−44%

Tokens saved on a SQL-cache hit. Memory's value is cost amortisation, not accuracy: conversation memory is +0.000.

53.3%

Out-of-sample match-outcome accuracy with no market data, against a 40.9% base rate and the bookmaker's 54.1%.

A documented, queryable data layer is the highest-leverage investment in an LLM analytics system. Multi-agent orchestration earns its place through error isolation and inspectable control flow — not through measured answer accuracy on single-step queries.

We report this as a null result deliberately: the comparison is controlled, the benchmark is frozen, and the cost difference is real.

Structured data access drives accuracy. Orchestration buys robustness, not accuracy points.

PART 1

Data & knowledge base

- Six providers with incompatible schemas, identifiers and grains
- A reproducible five-stage ETL pipeline
- Entity resolution and an explicit validation contract
- A three-layer knowledge base on managed PostgreSQL
- What the exploratory analysis implies for every downstream tool

Six providers, one authoritative store

6 data providers

WhoScored, Understat, FBref, Transfermarkt and Football-Data, plus a historical World Cup supplement. They differ in granularity, identifiers, access method and update frequency, and no two of them name a player or a club the same way.

1 authoritative store

The top five European leagues across five seasons, 2020-21 to 2024-25, consolidated into a single managed PostgreSQL instance. Every agent queries one source of truth rather than reconciling providers at runtime.

WHAT EACH SOURCE IS PRIMARY FOR

- **WhoScored** — match metadata and event actions: passes, shots, tackles, duels, set pieces and keeper actions, with timing, pitch coordinates and outcomes.
- **Understat** — expected goals and attacking quality, at shot, player-match and player-season grain.
- **FBref** — player and goalkeeper statistics, line-ups and formations, and the team-season context behind them.
- **Transfermarkt** — dated observations: player profiles, market-value snapshots, transfers and injury spells.
- **Football-Data** — pre-match bookmaker odds, and a second opinion on match statistics we already hold.

WHY NOT JUST ONE SOURCE

No provider covers the space on its own. Understat is the only source of expected goals; Transfermarkt the only source of market values and injuries; WhoScored the only source of event-level actions.

Where two providers do overlap, we keep both deliberately and use one to validate the other — which is most of what Football-Data is for.

The database is deliberately broader than any single analytical task.

Three failure modes: identity, grain, time

01 Identity

Six providers means six identifier spaces. The same player is spelled, transliterated and abbreviated differently in each of them, and clubs arrive under sponsorship names, abbreviations and historical variants.

HOW IT FAILS

Unresolved, one player becomes two entities, and every total for that player is split between them..

02 Grain

Eight different fact grains live in the same database: one row per match, per event, per player-match, per player-season, per team-season, per dated market value, per transfer and per injury spell.

HOW IT FAILS

Join two of them naively and one match row becomes one row per event, and every aggregate above it inflates.

03 Time

Market values, transfers, injuries, form and rest are dated observations rather than fixed attributes. What each of them is worth depends entirely on when you ask the question.

HOW IT FAILS

A rolling feature that includes the current match has already seen the result it is meant to predict.

None of the three raises an error. All three return a plausible number, which is why they are solved once, at ingestion, and never left to the agent.

Integration, not volume, is the hard part of this dataset.

Five stages from raw records to serving tables

1 **Extraction and raw staging**

Every source written unchanged to an immutable raw layer, with provenance: source, competition, season, extraction date, pipeline version.

2 **Cleaning and transformation**

One naming convention, real temporal types, canonical competition and season codes; nested event qualifiers decomposed and odds converted to implied probabilities.

3 **Entity resolution**

Deterministic alias dictionaries for clubs; normalised names, transliteration and fuzzy comparison for players, cross-validated on birth year, nationality, club and season.

4 **Validation and loading**

An explicit schema contract checked on every frame, then idempotent upserts in dependency order.

5 **Serving tables and features**

Materialised views at a declared grain, with every time-varying variable aligned to the analytical date, rolling features are shifted so a match can never influence its own features.

WHAT THAT BUYS

A failed transformation replays from the raw layer without going back to the provider, and the loader can be rerun without ever duplicating a record.

When the database was repopulated from one league to five after the benchmark had already been frozen, 29 of 30 canonical queries reproduced their original answers exactly.

AND WHAT IT REFUSES

Ambiguous entity mappings are written to a review log rather than silently accepted. A wrong merge is invisible; a missing row is not.

Provider quirks are absorbed at ingestion, so they never reach the tables the agents query.

Preserved grains, six serving views

ONE ROW PER …

- match
- event
- player and match
- player, team and season
- team and season
- dated market value
- transfer
- injury spell

Joining a match directly to its event records multiplies the match row by the number of events in it; a player-season joined to transfers becomes several rows for one player-season. Task queries therefore aggregate or filter each table down to a single grain before combining it with another.

PRINCIPAL SERVING VIEWS

Serving relation	Purpose
`vw_match_summary`	Metadata, scores, results, formations, odds
`vw_player_match_summary`	Player-match performance in match context
`vw_player_season_summary`	Season production, profile, market values
`vw_player_shooting_profile`	Shot locations, outcomes, body part, situation
`vw_player_availability`	Injury history, days and games missed
`vw_team_season_summary`	Standings, home-away splits, squad stats

Agents hold read-only access, and generated statements are checked before execution: anything that would modify data, or reach a relation the specification does not expose, is rejected.

The agent queries a documented relation at a declared grain. It never reconstructs the join itself.

Preserving grain is what stops a join from silently multiplying rows.

What the data forces on the rest of the system

01 Preserve the three classes

Home wins, draws and away wins are not equally common, and draws are roughly a quarter of fixtures. Draws cannot be dropped because they are hard to separate, and accuracy alone cannot be the measure of a forecast.

PART 3 · base-rate benchmark and Brier score, never accuracy alone

02 Represent uncertainty, not labels

Most matches finish with two or three goals, so a single action changes the result. A tool should return a probability distribution and the evidence behind it rather than one deterministic outcome.

PART 3 · calibrated distributions rather than an argmax label

03 Availability gates tool eligibility

Completeness varies by information family: cards, fouls, corners and formations are near-complete, while results, odds and shot data sit lower. A capability built on a thinner family has to declare its inputs.

PART 2 · a tool declares what it needs and refuses what it cannot answer

04 Align to the analytical date

Market values, transfers, injuries, form and rest use only observations preceding the target event, and rolling variables are shifted so a match can never influence its own features.

PART 3 · pre-kickoff features, and a guard that refuses incomplete vectors

Four findings from the exploratory analysis, each one a constraint the rest of the system had to obey.

The schema is the context window

A MACHINE-READABLE YAML SPECIFICATION

- Every relation documents its purpose, observational grain, primary and foreign keys, permitted join paths, temporal meaning and known caveats.
- It is injected into agent context, so no agent has to reverse-engineer the database at query time.
- It is maintained alongside the SQL schema: any change to a table, view, key or column lands in both, which is what stops the agents working from a stale picture.

FUNCTION 1 · GROUNDED SQL GENERATION

A semantically explicit description of the database rather than a list of table and column names: the model is told what a relation means and how it may legally be joined.

FUNCTION 2 · EVALUATION SUPPORT

The expected grain and the valid join paths feed the rule-based checks, so the Eval agent can flag an answer that reads fluently but was built on the wrong grain.

3 layers: raw ingestion, canonical entities, query-serving

0 write permissions held by any agent, by design

1 specification both the database and the agents are held to

Part 2 holds this layer constant, and asks whether the way we arrange agents on top of it changes anything.

A documented schema specification did more for accuracy than any orchestration change.

PART 2

System & experiments

- The LangGraph orchestration graph and what each agent does
- A frozen, content-hashed golden set and how answers are scored
- Three degrees of Data Analyst autonomy, and the three tools
- Results on autonomy
- Two single-LLM baselines, built to isolate orchestration from tool access
- Results on orchestration versus tools and database access

What it runs on

ORCHESTRATION

LangGraph

Defines the agent graph, manages routing between nodes and propagates shared state.

MODEL ROUTING

LiteLLM

Normalises each provider's request and response format, including function calling, into one provider-agnostic interface.

FOUNDATION MODELS

GPT-5.5 · Claude Opus 4.8 · Gemini 3.1 Pro Preview

Commercial APIs from OpenAI, Anthropic and Google, swapped by configuration rather than code.

OBSERVABILITY

Langfuse

Attached as a LiteLLM callback; the source of the per-run token and latency figures reported here.

ANALYTICAL DATA

PostgreSQL on Aiven, via SQLAlchemy

Queried by the Data Analyst behind the read-only guardrail.

MEMORY

pgvector on the same instance

Conversation history, SQL cache, corrections and documents; embeddings generated locally with all-MiniLM-L6-v2.

INTERFACE

Chainlit

Built-in chat front end for the conversational interface.

DEPLOYMENT

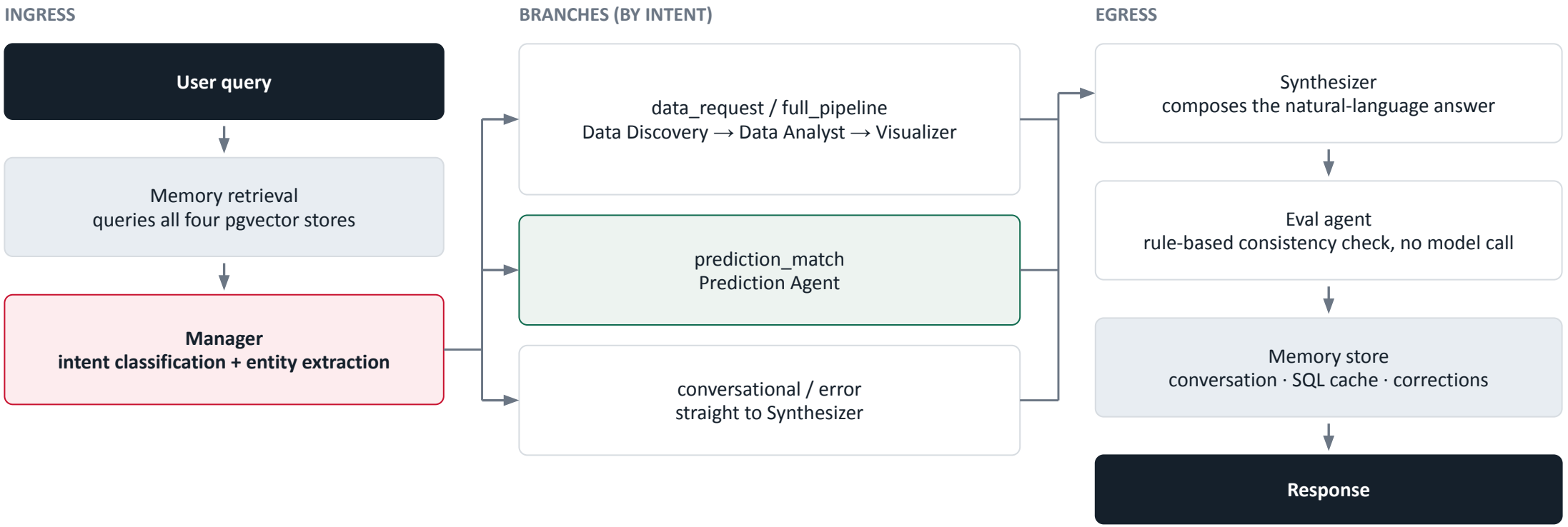
Modal

Serverless hosting for the application.

One interface is what makes the three-model autonomy sweep possible at all — the same pipeline runs under each provider without code changes.

Open-source frameworks with strong developer adoption, and models swappable by configuration.

LangGraph orchestration: nodes, edges, shared state



- Nodes are agents, edges define control flow, and one shared state object is passed between them — which is what makes the pipeline explicit and inspectable.
- A bounded retry loop grants two attempts before capping effort, containing worst-case cost.
- The Eval agent adds no model calls; detected errors are written to the correction store for reuse.

Conditional routing out of the Manager gives every query a well-defined, inspectable path.

What each agent is actually told

SHARED PREAMBLE · ALL AGENTS, ALL CONDITIONS

```
You are a Premier League data analyst. Answer the user's
question using ONLY the database. Rules: seasons are
'YYYY-YYYY' (hyphen, e.g. '2023-2024'); the latest season
is 2024-2025 (no 2025-26 data); point deductions are
already in the points columns; prefer IDs over names for
joins; canonical team names (e.g. 'Manchester United',
'Wolves').
```

DATA ANALYST · AUTONOMOUS SELF-DEBATE

```
# Candidate A
{preamble} + {schema}
Question: {question}
Write the most direct correct query.
Return ONLY one SQL SELECT statement.

# Candidate B
... Write an independent query (different phrasing/joins)
    for the same answer.

# Reconcile round (only if A and B disagree)
... Your two previous SQL attempts DISAGREED on their
    results -- reconcile them. Find the correct query.
```

ROLES

Manager	Classifies intent and extracts structured parameters as JSON — seasons, teams, metrics, and for forecasts the home team, away team and match date. Resolves “this season” to 2024-2025.
Data Discovery	Returns the relevant tables and short join/grain guidance for the question, given the schema and the Manager's output.
Data Analyst	Generates SQL behind a read-only guardrail. Two independent candidates per iteration, up to two reconcile rounds; if the result sets still disagree the answer is flagged low-confidence.
Visualizer	Plotly charts, invoked only on the full_pipeline branch.
Synthesizer	Composes the answer from the query result alone, stating numbers exactly.
Eval agent	Rule-based consistency check against the retrieved figures. No model call. Inconsistencies are flagged and written to the correction store.

Every condition shares this preamble and this schema context, so the comparison is fair by construction.

A frozen golden set, scored programmatically

30 data-analysis questions

Nine categories — league outcomes, player and team season aggregates, home/away splits, xG, discipline, transfers and market values, availability, match events, edge cases — across four difficulty tiers from simple lookups to multi-step edge cases.

30 prediction questions

Real fixtures from August 2024 to May 2025: 10 home wins, 9 draws, 11 away wins. Scored on the Brier score of the predicted H/D/A probabilities and on fidelity — whether the agent delegated to the model and reproduced its output.

HOW AN ANSWER IS SCORED

- Each data-analysis question is paired with a hand-verified canonical SQL query whose execution yields the reference answer.
- Questions, queries and results are content-hashed and frozen together with the originating git commit, so the agents never see the ground truth and the benchmark cannot drift between runs.
- At evaluation time every question is replayed through each configuration: exact match with a small numeric tolerance for scalars, row-set comparison for tabular and ranked answers.
- An LLM-as-judge from a different model family is available for genuinely free-text answers, but was not needed here — every item resolves to a deterministic SQL result or a numeric forecast score.

TWO EVALUATION SETS

The head-to-head MAS-versus-single-LLM comparison uses this re-scoped 60-question set on the current multi-league database. The autonomy experiment and the memory experiments were run earlier on the original 46-question single-league set and are reported on that basis. The two are not directly comparable — read them side by side rather than pooled.

Content-hashing the ground truth means any change in results reflects the system, not a moving target.

A frozen golden set, scored programmatically

ID	Category	Difficulty	Cov	Question
Q01	team_league_outcomes	simple	in	Which team won the Premier League in 2024-2025?
Q02	team_league_outcomes	simple	in	Points Arsenal finished 2023-2024 with?
Q03	team_league_outcomes	medium	in	Full 2022-2023 PL final table (rank, team, points).
Q05	team_league_outcomes	medium	in	Best goal difference in 2024-2025?
Q06	team_league_outcomes	edge	out	Everton 2023-2024 points with vs without deduction?
Q07	team_league_outcomes	hard	out	Highest single-season points total, 2020-2025?
Q09	player_season_aggregates	simple	in	Top goalscorer in 2023-2024 and goals?
Q10	player_season_aggregates	medium	in	Top five goalscorers in 2024-2025 with totals.
Q11	player_season_aggregates	medium	in	Most assists in 2023-2024?
Q12	player_season_aggregates	edge	in	Cole Palmer total 2023-2024 goals (all clubs)?
Q16	home_away_splits	medium	in	Most home points in 2024-2025?
Q18	home_away_splits	hard	out	Biggest home-vs-away points gap in 2023-2024?
Q19	home_away_splits	simple	in	Arsenal home points in 2023-2024?
Q20	home_away_splits	medium	in	Most home goals in 2024-2025?
Q21	disciplinary	medium	in	Most yellow cards (team) in 2023-2024?

Three degrees of autonomy, three tools

Fully controlled

Answers only by selecting from a catalogue of predefined, parameterised SQL templates. One corrective retry on template error.

Semi-controlled

Has the templates and may also write its own SQL, deciding per query — and falls back to free SQL if a chosen template fails.

Fully autonomous

Writes every query itself: two independent statements, reconciled before answering. This is the variant carried into Experiments 2 and 3.

The semi-controlled action space is a superset of both others', so we hypothesised it would be at least as accurate as either. That superset holds over available actions, not outcomes — the extra template-or-SQL routing decision is a new source of error.

TOOLS, EXPOSED AS CALLABLE FUNCTIONS RATHER THAN BACKEND ACCESS

`use_template`

A catalogue of parameterised SQL covering the most common analytical questions: league table, team-season summary, home/away splits, player and team stat leaders, head-to-head, market values, transfers.

`run_sql`

Free-form SQL against the football database, behind a read-only validation layer that rejects any non-SELECT statement.

`predict_match_outcome`

Wraps the trained model: given home team, away team and date it returns calibrated home / draw / away probabilities, computed only from data available before kickoff.

A larger action space does not guarantee better outcomes: the routing decision is itself a place to fail.

Experiment 1: how much autonomy should the analyst have?

Model	Variant	In-coverage	Out-of-coverage	Overall
Opus 4.8	Controlled	26/28 (93%)	1/18 (6%)	27/46 (59%)
Opus 4.8	Semi	26/28 (93%)	16/18 (89%)	42/46 (91%)
Opus 4.8	Autonomous	24/28 (85%)	17/18 (94%)	41/46 (89%)
GPT 5.5	Controlled	26/28 (93%)	3/18 (17%)	29/46 (63%)
GPT 5.5	Semi	25/28 (89%)	16/18 (89%)	41/46 (89%)
GPT 5.5	Autonomous	23/28 (82%)	16/18 (89%)	39/46 (85%)
Gemini 3.1 Pro Preview	Controlled	26/28 (93%)	2/18 (11%)	28/46 (61%)
Gemini 3.1 Pro Preview	Semi	26/28 (93%)	16/18 (89%)	42/46 (91%)
Gemini 3.1 Pro Preview	Autonomous	19/28 (68%)	13/18 (72%)	32/46 (70%)

93%

In-coverage accuracy of the controlled variant — identical across all three models. Templates are reliable where they apply.

6–17%

Controlled variant out of coverage. Templates cannot express computed columns, HAVING filters or cross-season logic.

70%

Gemini's autonomous variant overall — the self-debate loop repeatedly failed to converge, where Opus held at 89%.

The same pipeline is production-ready under one foundation model and unreliable under another.

Two baselines, built to separate two effects

SINGLE LLM + TOOLS

Native function-calling access to the same two tools the autonomous MAS uses — run_sql and predict_match_outcome — plus the identical schema context. Answers by issuing tool calls in a loop, up to eight steps. No orchestration.

Isolates the effect of orchestration alone.

SINGLE LLM + RAW DATA

No tools and no database access. A text dump of the core modelling tables — season standings, home/away splits, team and player season statistics — is inlined directly into the system prompt, with instructions to answer only from that data and say so when it is absent.

Isolates data reasoning from data retrieval.

WHY INLINE THE DATA RATHER THAN GIVE IT RETRIEVAL TOOLS

- To separate the model's data-reasoning capability from any retrieval or external-knowledge capability, and because a fixed inlined dataset makes runs deterministic and directly comparable across models and architectures.
- The cost is a hard coverage ceiling: by design and by context size the dump excludes match events, player-match performance, transfers, market values and injuries. Only 20 of the 30 evaluation questions are answerable in this condition at all.
- Read its accuracy accordingly — the shortfall is missing source data, not failed reasoning. We return to this on the results slide.

Comparing the three conditions separates the gain from tool access from the gain from orchestration.

Experiment 2: does orchestration add accuracy?

Condition	Overall	Avg tokens	Avg latency
MAS (autonomous)	26/30 (87%)	32,324	16.7s
single-LLM + tools	26/30 (87%)	22,150	8.1s
single-LLM + raw data	21/30 (70%)	121,711	4.6s

Condition	Avg Brier	Fidelity	Avg tokens	Avg latency
MAS (autonomous)	0.197	30/30	1,008	6.8s
single-LLM + tools	0.197	30/30	42,912	13.0s
single-LLM + raw data	0.196	0/30	1,204	7.7s

p = 1.0

McNemar exact test on the paired binary outcomes. Only four discordant questions: b = 2 MAS-only correct, c = 2 single-only.

20/20

Raw-data score on the questions whose source tables fit in the prompt. The other 10 are unanswerable by construction.

HOW TO READ THIS

- The 70% → 87% gap is not evidence that tools improve reasoning. The raw-data condition scores 20/20 on answerable questions; its shortfall is entirely the 10 questions whose tables are too large to inline.
- That is a context-window scaling limit, so we frame it as data coverage rather than reasoning.
- The programmatic tie slightly flatters the MAS: two of the four discordant questions were formatting losses (a fee written as “116.6 million”, a correct denial broken by markdown).

Identical accuracy, 1.5× the tokens, 2× the latency. The tie is the finding.

PART 3

Memory & prediction

- Four pgvector stores, mapped onto a functional taxonomy of agent memory
- The SQL cache as a cost mechanism — and a safety failure it exposes
- A 55-session ablation isolating each store against a matched baseline
- Prediction as a first-class agent tool, decoupled from the graph
- A calibrated stacking ensemble, backtested on 379 held-out fixtures

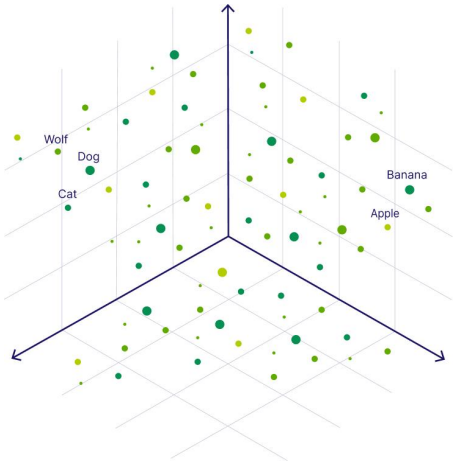
Memory architecture: four stores, four thresholds

		COSINE THRESHOLD
Factual memory *What does the agent know?*	`conversation_memory` keyed on the user question · written when a turn succeeds	0.70
Experiential memory *How does the agent improve?*	`sql_cache` · `correction_memory` case-based reuse of prior runs: successful SQL cached, failed patterns flagged	0.88 / 0.75
Working memory *What is the agent thinking now?*	`LangGraph state object` in-session only, discarded on response — deliberately not persisted	—
Knowledge base (RAG) *Outside agent memory proper*	`documents` content-keyed, populated by offline ingestion; agents never write to it	0.30

Thresholds are set by risk: a cache hit answers the query, so it needs a far higher bar than context.

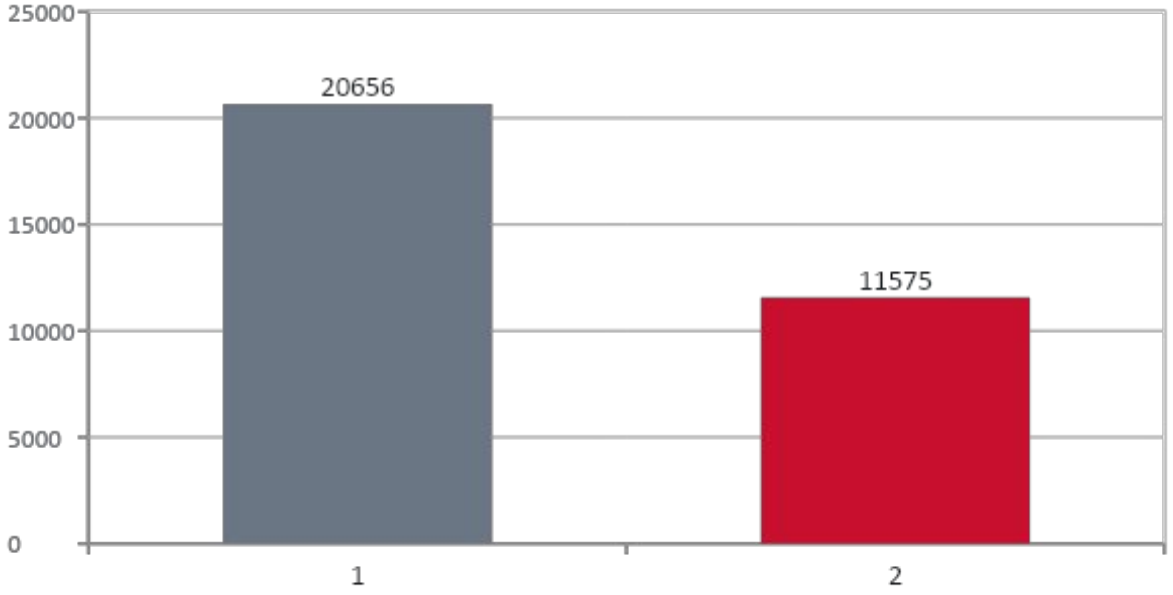
SHARED MACHINERY

- all-MiniLM-L6-v2, 384 dimensions, executed locally.
- One encoder for all stores, so a single question embedding serves every lookup in a turn.
- One retrieval node at the graph entry point, before intent classification.
- One terminal store node after Eval; every write is wrapped so a storage failure cannot break the turn.



The SQL cache is a cost mechanism

Average tokens per query, 46-question sweep at 0.88 cosine threshold



−44% tokens on a cache hit

−17% latency: 14.44 s → 11.97 s

WHAT THE SWEEP DID

- Every question is run cold to establish a cache-free baseline, then re-run three ways: exact repeat, light reword, and near miss (swapped player or season).
- Exact repeats hit 45 times out of 46 at mean similarity 0.998, with no material accuracy loss.
- Light rewords hit 44 of 46 at 0.983 — but accuracy falls to 65%, with three answers drifting.
- A near-identical hit rate with worse accuracy is the warning sign: a paraphrase-level embedding match does not guarantee the reused SQL still answers the reworded question.
- The threshold gives no signal here. Reword similarity (0.983) sits just below exact repeats (0.998) and far above the 0.88 cut-off, so it admits a drifted match with almost the same confidence as a true one.

The saving comes from skipping SQL generation entirely, so it scales with how often a workload re-asks.

Near misses: cosine similarity is unsafe for temporal queries

Tier	Cache hits	Accuracy	Mean similarity	Latency	Tokens
Cold baseline	0 / 46	72% (33/46)	—	14.44 s	20,656
Exact repeat	45 / 46	72%	0.998	11.97 s	11,575
Light reword	44 / 46	65% (3 drifted)	0.983	12.12 s	11,339
Near miss	**43 / 46**	**~7% (collapse)**	**0.978**	**12.10 s**	**11,606**

0.978 vs 0.88

A query with the same surface form but a different team or season embeds at 0.978 — comfortably above the 0.88 threshold. It retrieves the cached result 43 times out of 46 and returns a confidently wrong answer with no error signal.

WHY THRESHOLD TUNING IS NOT THE FIX

- Raising 0.88 → 0.99 would suppress near misses, but also kill the legitimate rewords the cache exists to serve.
- The failure is categorical, not gradual: season and entity identity are exactly what a sentence embedding compresses away.
- We report this as a safety finding rather than a bug: any similarity-keyed cache over temporal data inherits it.

Semantic similarity is good at finding candidate matches and bad at deciding they are safe to reuse.

The fix, and what memory actually buys

TWO-STAGE REUSE GATE, NOW DEPLOYED

1 Similarity

Nearest cached question must clear the 0.88 cosine threshold.

2 Entity consistency

Discriminating tokens — season, metric family, named entities — are extracted from the incoming query and compared against the cached query and its stored SQL.

3 Reject on contradiction

Any disagreement rejects the hit. A cheap textual comparison, no extra model call, negligible latency.

Known gap: it does not catch polarity swaps ("highest" vs "lowest") that leave season, metric and entity unchanged. The near-miss sweep has not yet been re-run under the guard.

SESSION-BASED ABLATION: 55 SESSIONS, MEMORY ON VS OFF

Store	Sessions	Δ acc.	95% CI	Wilcoxon
Correction memory	20	**+0.200**	[0.05, 0.40]	p = 0.125
Conversation memory	17	+0.000	[−0.176, 0.162]	p = 1.0
SQL cache	18	+0.000	—	—

- Correction memory, by mechanism: the induced failure recurred in 5 of 20 no-memory sessions — all the same referee column containing a non-breaking space, so a naive equality filter returns zero rows silently. In 4 of those 5 the memory arm had written a normalising pattern and reapplied it.
- Conversation memory's zero is exact cancellation, not inertness: 5 sessions improved, 5 degraded, 7 unchanged.
- Against the baselines it is clear-cut: +0.441 over a stateless single LLM (p = 0.0005), but −0.559 against one given the full history in context — which is far more token-expensive.
- p = 0.125 is a hard floor at n = 4 non-zero deltas, not a tunable number. Read the interval and the mechanism alongside it.

Memory earns its place on cost, and on real data quirks a fixed cleaning pass would never anticipate.

Prediction as a first-class agent tool

CAPABILITY SELECTION CASCADE

1 **Manager task ID resolves in the registry**

Run the manager's capability.

2 **Otherwise, match the keyword-cue table**

Team names paired with matchup language, checked against the raw query text.

3 **Otherwise, fixed default**

Match outcome / season class. Degrades gracefully when the manager's free-text output is malformed.

Three steps rather than one lookup, so there is always a traceable reason why a given capability fired.

13 forecasting capabilities available at inference

6 method families, from calibrated ensembles to empirical frequency

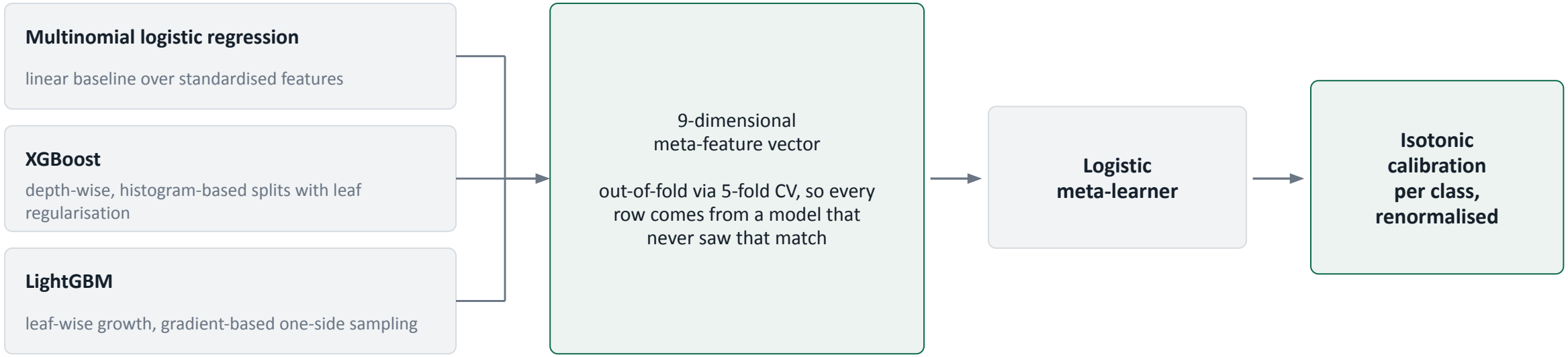
1 family with calibrated probabilities and external baselines: match outcome

DESIGN CONSTRAINTS

- Training on the fly is neither time- nor cost-efficient, so capabilities load pre-trained artefacts at inference.
- The forecast is written into the same state field the Data Analyst would populate, so Synthesizer and Eval need no prediction-specific handling.

The package knows nothing about LangGraph; the agent node is its only point of contact.

The match-outcome model: stacked, then calibrated



PRE-KICKOFF FEATURES ONLY

- Elo team strength; rolling five-match form (points, goals, shots, xG for and against); rest days; team-level xG aggregated from player data.
- Two variants: with odds for offline analysis, no odds for live prediction.

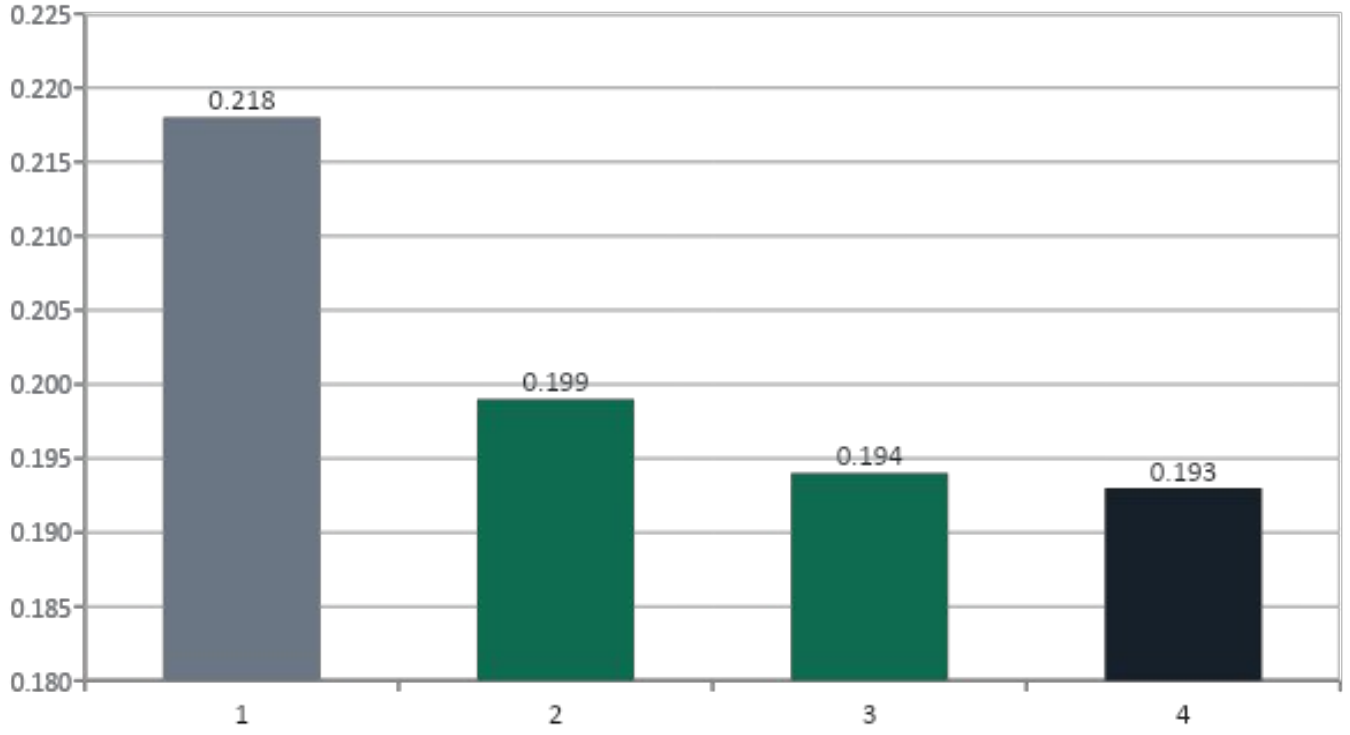
TRAIN / INFERENCE PARITY

- A synthetic fixture is appended to the historical record and pushed through the identical pipeline, so rolling and xG features reflect the true pre-match state.
- A runtime guard refuses any incomplete feature vector rather than imputing it.

Calibrated, not merely ranked — a fan comparing our numbers to the odds acts on the magnitude.

Out-of-sample backtest: 379 held-out fixtures

Brier score, lower is better · trained on 2020–2024, tested on 2024–2025 (n = 379)



+12.4 pts

Accuracy over the base-rate baseline: 53.3% vs 40.9%. A bootstrap 95% CI on the no-odds model contains the baseline's 54.1%.

+0.3 pts

All that adding baseline odds buys. The engineered features already carry most of the signal the odds encode.

−9.4 pts

Accuracy lost when Elo is group-wise permuted, against −2.5 for defensive form and −1.2 for attacking form. Team strength dominates.

Engineered features recover most of the market's win/draw/loss information without using the market.

Delegation works

Condition	Avg Brier	Fidelity	Avg tokens	Avg latency
MAS (autonomous)	0.197	**30 / 30**	1,008	6.8 s
Single LLM + tools	0.197	**30 / 30**	42,912	13.0 s
Single LLM + raw data	0.196	**0 / 30**	1,204	7.7 s

- Both tool-using conditions delegated on all 30 fixtures and reproduced the forecast vector faithfully.
- Their outputs are identical by construction: single-pass parameter extraction, then the same function call. The prediction slice admits no architectural comparison and no significance test.
- The single LLM with tools reaches that identical output at 42× the tokens, exploring the database before delegating.

42× the tokens, for a byte-identical forecast

STATED BEFORE WE ARE ASKED

- A single LLM reasoning with no model at all scores 0.196 — marginally better. On 30 fixtures that is statistically indistinguishable.
- The fixtures are public Premier League results the language model may partly recall, so the comparison is not clean.
- Results are confined to one Premier League season. A larger, less memorisable test set is needed to establish the model's added value over free-text estimation.

The contribution is integrating a calibrated model as a first-class tool, not beating free-text estimation.

Limitations, in order of how much they bound the claims

01

Task scope does not exercise orchestration

Every benchmark question maps to one retrieval and one answer. Multi-agent systems are theorised to help on compositional, multi-step tasks. Our null result is specific to single-step querying.

02

Tool access and data coverage are confounded

The 70% ⟶ 87% gap is a context-window scaling limit, not proof that tools improve reasoning. A retrieval or text-to-SQL baseline with equal coverage would isolate it.

03

A static pipeline, not a dynamic MAS

Fixed graph, largely sequential. No parallel same-role agents, no runtime re-routing beyond the bounded retry. Says nothing about decentralised architectures.

04

Small benchmark, single run

30 questions, 30 fixtures, one run per condition, non-zero temperature. McNemar rests on 2–4 discordant pairs, so it cannot exclude a small effect.

05

Database substrate changed after freezing

Repopulated from single-league to five-league after the benchmark froze. 29 of 30 re-scoped canonical queries reproduce the original answers; the autonomy experiment ran on the old substrate.

The first two are scope conditions on the headline result, not footnotes.

Engineering defaults we would ship

01 **Invest in the data layer first**

Clean, documented, queryable structured access is the main source of accuracy. A YAML schema specification is worth more than fancier orchestration.

02 **A single LLM with tools is enough for predictable loads**

It matched the MAS at 87% on a third fewer tokens and half the latency. Reach for orchestration when you need error isolation and inspectable control flow on ambiguous, unseen queries.

03 **Give the analyst autonomy, but pin the model**

Free SQL is what answers out-of-coverage questions. Reliability varies sharply by foundation model, so revalidate before swapping providers on cost.

04 **Treat memory as a cost mechanism**

Cache reuse amortises repeated work; correction memory catches real data quirks. Gate similarity reuse on entity and season consistency, or it will serve the wrong year confidently.

05 **Communicate calibrated distributions, not argmax labels**

Users act on the stated confidence. A model that cannot predict draws should not be presenting a single most-likely outcome.

Open question next: whether orchestration earns its cost on genuinely multi-step, compositional queries — the regime this benchmark does not exercise.

Backup

Model configuration · exact prompts · golden-set questions · prediction fixtures

BACKUP · A1

Model identifiers and run configuration

Logical family	LiteLLM model ID	Temperature
claude	anthropic/claude-opus-4-8	model default (omitted)
gpt	gpt-5.5	model default (omitted)
gemini	gemini/gemini-3.1-pro-preview	model default (omitted)

- All calls route through LiteLLM, giving one OpenAI-style interface (including cross-provider function calling) across three families.
- All three models deprecate or restrict temperature, so we omit it. SQL paths request 0.3 but it is suppressed — a methodological constraint, not a fixed temperature.
- max_tokens 4096, generous enough that thinking models return content rather than only reasoning.
- Memory disabled for all Experiment 2 runs, so the comparison isolates orchestration.

INSTRUMENTATION CAVEATS

- Tokens and latency captured locally via LiteLLM usage accounting — except MAS data-analysis token totals, recovered from Langfuse traces because local capture does not see the analyst's self-debate loop. The two are not directly comparable across tables.
- Single run per condition, no variance estimates: differences of one to three questions sit within noise.
- Q41 and Q25 ground truth should be re-audited (grain mismatch; non-breaking-space normalisation).

BACKUP · A2

Exact prompts per condition

MAS MANAGER · INTENT AND ENTITY EXTRACTION

```
Classify and extract for this Premier League question.
Question: {question}
Respond JSON: {"intent": "data_request" |
  "prediction_match" | "conversational",
  "seasons": ["YYYY-YYYY", ...], "teams": [...],
  "metrics": [...], "home_team": "...",
  "away_team": "...", "match_date": "YYYY-MM-DD"}
Use intent='prediction_match' ONLY when asked to forecast
a specific fixture's outcome. Resolve 'this/current
season' to 2024-2025 (latest data).
```

SINGLE LLM + RAW DATA · PREDICTION VARIANT

```
You have NO tools and NO prediction model -- forecast
using ONLY the recent results below. Reason about form,
then give CALIBRATED probabilities for {home_team} (home)
vs {away_team} on {match_date} that sum to 1.0.

=== RECENT RESULTS ===
{last 10 matches strictly before match_date}

End your answer with EXACTLY this line:
PROBABILITIES: home=<p>, draw=<p>, away=<p>
```

TEMPLATE CATALOGUE · CONTROLLED AND SEMI VARIANTS

```
league_table                (season)
team_season_summary         (team, season)
home_away_table             (season, metric)
player_season_leaders       (season, metric)
player_team_leaders         (team, season, metric)
team_stat_leaders           (season, metric)
head_to_head                (team_a, team_b, season)
match_results               (season)
market_value_leaders        (-)
market_value_team_leaders   (team)
player_match_leaders        (season, metric)
transfer_leaders            (-)
```

TOOL SCHEMA SHOWN TO THE MODEL

```
name: predict_match_outcome
description: Predict probabilities of each outcome
  (home win / draw / away win) for a Premier League
  fixture, using the trained match-outcome model.
  Returns three probabilities summing to 1.0 plus the
  model's confidence. Provide the match_date (the model
  is causal -- it uses only data before that date).
input: { home_team (req), away_team (req),
         match_date 'YYYY-MM-DD' (req), season (opt) }
```

BACKUP · A3

Golden-set data-analysis questions (sample)

ID	Category	Tier	Cov	Question
Q01	team_league_outcomes	simple	in	Which team won the Premier League in 2024-2025?
Q03	team_league_outcomes	medium	in	Full 2022-2023 PL final table (rank, team, points).
Q06	team_league_outcomes	edge	out	Everton 2023-2024 points with vs without deduction?
Q12	player_season_aggregates	edge	in	Cole Palmer total 2023-2024 goals (all clubs)?
Q18	home_away_splits	hard	out	Biggest home-vs-away points gap in 2023-2024?
Q25	disciplinary	edge	out	PL matches refereed by Anthony Taylor in 2023-2024?
Q26	xg_performance	hard	out	Biggest xG overperformer (≥10 goals) 2023-2024?
Q41	event_player_match	medium	in	Most goals by one player in a single match, 2024-2025?
Q45	edge_case	edge	in	Norwich City points in 2024-2025? (expect denial)

- Nine of the thirty shown. IDs are inherited from the original 46-question set from which the benchmark was curated, so gaps reflect questions not retained.
- “Cov” is SQL-template coverage: whether the question can be answered by an available template or needs different query and join logic.
- The complete canonical SQL for every question is in results/golden/ground_truth_v2.json, content-hashed with the originating commit.

BACKUP · A4

Prediction fixtures and model ground truth (sample)

ID	Fixture	Date	Actual	Model H / D / A
P02	Arsenal vs Wolves	2024-08-17	H	0.800 / 0.184 / 0.016
P04	Nottingham Forest vs Bournemouth	2024-08-17	D	0.322 / 0.249 / 0.429
P07	Brentford vs West Ham United	2024-09-28	D	0.454 / 0.278 / 0.268
P10	Wolves vs Manchester City	2024-10-20	A	0.049 / 0.261 / 0.690
P14	Newcastle United vs Liverpool	2024-12-04	D	0.244 / 0.234 / 0.522
P20	Manchester City vs Chelsea	2025-01-25	H	0.605 / 0.211 / 0.184
P25	Southampton vs Crystal Palace	2025-04-02	D	0.049 / 0.222 / 0.729
P28	Wolves vs Leicester City	2025-04-26	H	0.673 / 0.190 / 0.137
P30	Ipswich Town vs West Ham United	2025-05-25	A	0.301 / 0.229 / 0.469

- Nine of the thirty shown. The full set spans 2024-08-16 to 2025-05-25 across 20 clubs, with a realised balance of 10 home wins, 9 draws and 11 away wins.
- Every row above with a drawn outcome shows the draw-underprediction limitation directly: under argmax the model never assigns the draw the highest probability on any of the nine drawn fixtures.
- Scored on the Brier score of the predicted vector against the realised outcome, and on whether the agent correctly delegated to the prediction model.